

Documento final · TPE Tema 9 · Protección de Servicios con WAF para The Store

Tema 9 · Redes de Información · ITBA · 1C 2026

Grupo 2

Integrantes

Mauro Vella

Enrique Castillo - 68321

Federico Inti Garcia Lauberer - 61374

Este documento es la **entrega final** del TPE y extiende la pre-entrega del 21/04. Mantiene el contexto y el diseño propuestos originalmente, y agrega lo que la pre-entrega no podía tener todavía: la **implementación realizada**, los **resultados reales** medidos sobre el cluster, los **problemas encontrados durante el tuning** y la **conclusión** con limitaciones honestas.

1. Problemática y contexto

The Store es una aplicación de e-commerce construida con **cinco microservicios** (Java/Spring, Go/Gin, Node.js/NestJS) que se despliega sobre un cluster local de Kubernetes (Kind) de un único nodo. Toda la entrada externa pasa por un único punto: `ingress-nginx` escuchando en el puerto 80/443 del host. Los backends (`catalog`, `carts`, `orders`, `checkout`) son `ClusterIP` y no deberían ser accesibles desde afuera; sólo la `ui` está publicada.

- La auditoría pre-WAF contra `http://localhost` confirmó **10 vectores explotables** sin autenticación, agrupados en **6 clases de hallazgo**. Los más críticos: SSRF / path traversal vía `/proxy/**`, acceso a rutas administrativas de backends vía el proxy, y exposición de Spring Actuator.
- No había **rate limiting**, **scanner detection** ni **headers HTTP de seguridad**. Esto dejaba la puerta abierta a enumeración, scraping, abuso automatizado, fuga de información operativa y reconocimiento.
- El problema es **transversal a una arquitectura multi-lenguaje**: corregir en Java, Go y Node por separado requiere cambios distribuidos. Un WAF aplica una **política unificada en el punto de entrada**, sin tocar el código de negocio.

1.1 Hallazgos de la auditoría (6 clases · 10 vectores confirmados)

ID	Clase de hallazgo	Severidad	OWASP 2025	Vectores (PoCs) confirmados
H1	SSRF + path traversal vía <code>/proxy/**</code>	Crítica	A03 / A10	<code>/proxy/carts/admin</code> , <code>/proxy/carts/test</code> , <code>/proxy/catalog/../../../../actuator/info</code> , <code>/proxy/catalog/../../../../actuator/prometheus</code> , <code>/proxy/orders/anything</code> , <code>/proxy/checkout/anything</code> (6)
H2	Inyecciones en checkout (XSS / SQLi / CRLF)	Alta	A03	XSS y SQLi en el form de checkout (cubiertos por CRS)
H3	Endpoint de chat sin validación / prompt injection	Alta	A04	<code>POST /chat/submit</code> (feature opcional)
H4	Spring Actuator expuesto	Alta	A05	<code>/actuator/prometheus</code> , <code>/actuator/info</code> , <code>/actuator/metrics</code> , <code>/actuator/health</code> (4)
H5	Confianza incondicional en <code>X-Forwarded-*</code>	Media	A07	spoof de IP/proto/host
H6	Sin rate limit / scanner detection / headers	Alta	A04 / A05	UA de scanners, burst sin límite, 0/6 headers

Los 10 PoCs de la baseline pre-WAF (todos `verdict: vuln`, 0 bloqueados, ~53 KB filtrados) están en [pre-analysis/evidencias/pocs-data.json](#).

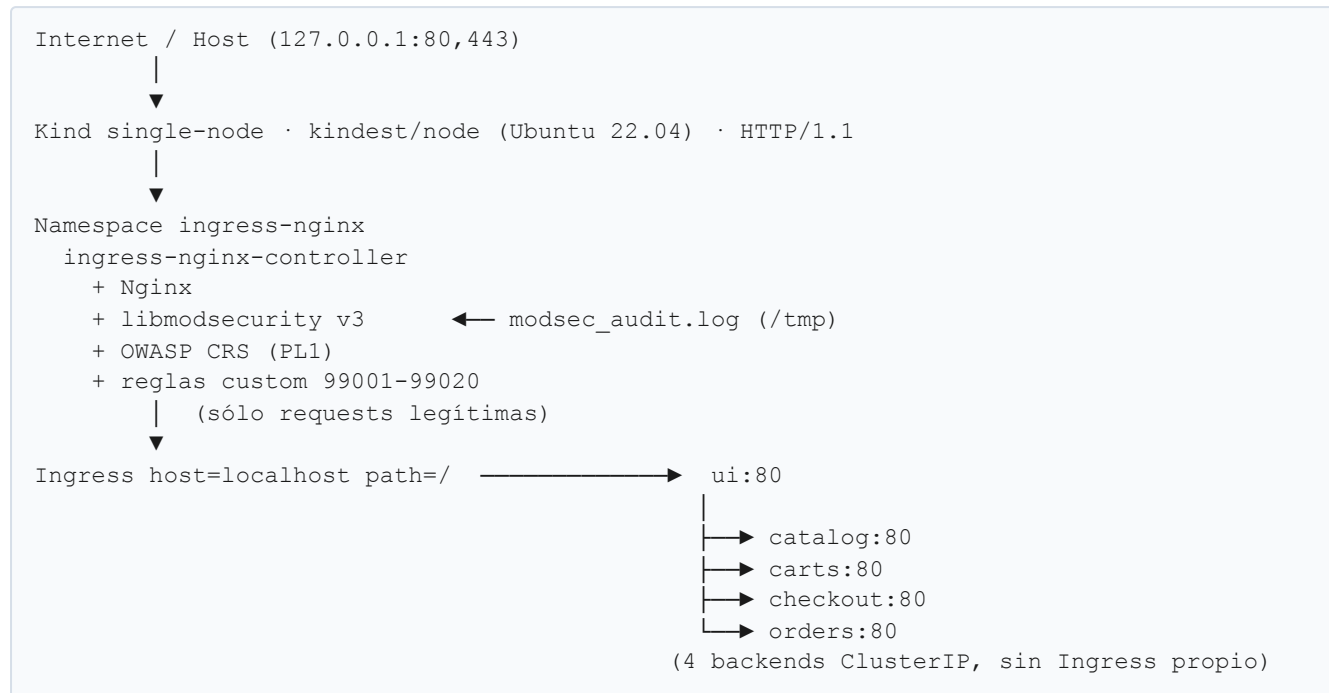
2. Diseño de la solución (recordatorio del diseño aprobado)

Se desplegó **ModSecurity v3 + OWASP Core Rule Set (CRS)** dentro del `ingress-nginx-controller` ya presente en el cluster, **sin modificar ni una línea de los microservicios**. La imagen oficial `registry.k8s.io/ingress-nginx/controller:v1.13.x` ya trae `libmodsecurity` y el CRS pre-compilados; se activan vía ConfigMap.

La política se compone de cuatro piezas:

- **ConfigMap del controller** — activa `enable-modsecurity`, `enable-owasp-modsecurity-crs`, define el motor (`SecRuleEngine On`), el audit log y el `Include` de las reglas custom.
- **OWASP CRS (Paranoia Level 1)** — cubre traversal, scanners e inyecciones genéricas de capa 7 (XSS, SQLi) sin tocar el código.
- **Reglas custom 99001-99020** — para la lógica específica de The Store: Actuator, traversal vía `/proxy/**`, rutas admin de backends y scanners.
- **Anotaciones del Ingress** — `limit-rps/limit-connections` (rate limiting), `more_set_headers` (6 headers de seguridad) y CSP en modo Report-Only.

3. Arquitectura de red



Bloque	CIDR	Rol
Host loopback	127.0.0.1/32	El usuario accede a The Store por 80/443
Docker / nodo Kind	172.18.0.0/16	Red del contenedor del nodo
Pods	10.244.0.0/16	Comunicación pod-a-pod
Services (ClusterIP)	10.96.0.0/12	IPs internas de los Services
CoreDNS	10.96.0.10	DNS interno del cluster

El WAF se inserta en el límite **norte-sur** (Internet → cluster). El tráfico **este-oeste** (entre pods, p. ej. `ui` → `catalog`) NO pasa por el WAF: es una limitación consciente del diseño (ver §7).

4. Implementación realizada

Todo el WAF vive en [deploy/waf/](#) y es **declarativo, idempotente y reversible**.

4.1 Manifiestos

Archivo	Qué hace
01-controller-configmap.yaml	Reemplaza la ConfigMap del controller: activa ModSec + CRS, fuerza 429 en rate limit, <code>server-tokens: false</code> , <code>use-forwarded-headers: false</code> (cierra H5), motor <code>SecRuleEngine On</code> , audit log en <code>/tmp/modsec_audit.log</code> (<code>SecAuditLogParts ABCFHZ</code>) e <code>Include</code> de las reglas custom.
02-the-store-ingress.yaml	Reemplaza el Ingress de <code>ui: rate limit</code> por IP (<code>limit-rps: 10</code> , <code>limit-connections: 20</code> , <code>limit-burst-multiplier: 3</code>), <code>proxy-body-size: 1m</code> , los 6 headers de seguridad vía <code>more_set_headers</code> , CSP Report-Only y refuerzo de inspección de body por-Ingress.
03-controller-deployment-patch.yaml	Strategic-merge patch que monta la ConfigMap <code>modsec-custom-rules</code> como volumen read-only en <code>/etc/nginx/modsecurity-custom/</code> .
rules/the-store.conf	Las 7 reglas custom (99001-99020).
install.sh / uninstall.sh	Instalador idempotente (con smoke-test) y revert al estado pre-WAF.

4.2 Reglas custom (rango 99000-99999, reservado por CRS para el usuario)

ID	Hallazgo	Acción	Detalle
99001	H4 · Actuator	deny 403	Bloquea <code>/actuator/{info,metrics,prometheus,env,heapdump,...}</code> directo y vía proxy, más el índice <code>/actuator</code> (que lista en HAL todos los endpoints disponibles). Excluye <code>health</code> (lo usan los probes de K8s).
99002	H1 · traversal	deny 403	<code>/proxy/*</code> con <code>..</code> , <code>%2e%2e</code> o <code>%252e%252e</code> (doble URL-encode).
99003	H1 · admin vía proxy	deny 403	<code>/proxy/<svc>/(actuator\ debug\ management\ admin)</code> .
99004	H1 · proxy sin sesión	deny 403	<code>/proxy/*</code> con cookie <code>SESSIONID</code> presente pero sin formato UUID válido (chain).
99005	H1 · proxy sin sesión	deny 403	<code>/proxy/*</code> sin ningún header <code>Cookie</code> (chain con <code>&...@eq 0</code>). Cubre el acceso directo por curl/scanner, que 99004 no ve porque libmodsecurity v3 no itera sobre una colección ausente.
99010	H6 · scanners	deny 403	User-Agents de <code>sqlmap</code> , <code>nikto</code> , <code>nuclei</code> , <code>nmap</code> , <code>wpscan</code> , ... (refuerza CRS 913xxx).
99020	tuning	pass	Whitelist quirúrgica: desactiva CRS 920350 (Host numérico) sólo para <code>Host: localhost/127.0.0.1</code> .

Reglas 99004/99005 · la segunda rama del cierre del proxy. La pre-entrega comprometió denegar `/proxy/*` que tuviera `..` o que "no trajera header de sesión válido". La primera rama la cubre 99002; estas dos cierran la segunda. La app emite una cookie `SESSIONID` (UUID) en la primera visita a `/`

(`SessionIDWebFilter`), y el front **nunca** invoca `/proxy/*` desde el navegador — así que exigir la cookie no rompe ningún flujo legítimo, sólo corta el acceso directo por curl/scanner. Limitación honesta: un atacante puede fabricar una `SESSIONID` con formato UUID; la validación criptográfica de la sesión es responsabilidad de la app, no del WAF. La regla eleva la barrera y cubre el caso por defecto.

Las inyecciones de H2 (XSS/SQLi) **no necesitaron regla custom**: las cubre el CRS base (familias `941xxx`, `942xxx`, `scoring 949110`). Esto valida la decisión de diseño: **CRS para lo genérico + reglas custom para lo específico de la app**.

4.3 Integración en `local.sh`

El WAF dejó de instalarse "aparte": se integró en el orquestador raíz siguiendo el estilo existente (`cmd_*`, `print_status`, verificación de prerequisites):

- `./local.sh install-waf` y `./local.sh uninstall-waf` — wrappers que validan que el cluster y el namespace existan antes de delegar en `deploy/waf/install.sh` (o `uninstall.sh`), pasando el namespace correcto vía `APP_NS`.
- `./local.sh create-cluster --with-waf` — levanta el cluster, despliega la app y deja el WAF instalado en un solo comando. El WAF se instala **después** de los e2e tests a propósito: el rate limit (10 rps/IP) podría throttlear el tráfico legítimo de la suite.

5. Resultados (medición real sobre el cluster)

Corrida end-to-end real sobre Kind. Baseline pre-WAF y verificación post-WAF capturados el mismo día contra `http://localhost`. Evidencia versionada en [pre-analysis/evidencias/post-waf/](#) (incluye el audit log de ModSecurity).

5.1 Tabla pre/post de los 5 casos comprometidos

#	Caso	Pre-WAF	Post-WAF	Qué lo bloqueó
1	<code>GET /actuator/prometheus</code>	200 OK · ~19 KB filtrados	403	regla custom 99001
2	Path traversal <code>/proxy/catalog/../../../../actuator/info</code>	200 OK	403	regla custom 99002
3	<code>User-Agent: Nikto / sqlmap / nuclei</code>	200 OK	403	regla custom 99010
4	Burst de 100 requests concurrentes a <code>/</code>	100/100 → 200	23/100 → 429	nginx <code>limit_req</code> (10 rps + burst×3)
5	Headers de seguridad en la home	0/6 presentes	6/6 presentes	annotation <code>more_set_headers</code>

Los 5 casos pasan de vulnerable a mitigado, y el tráfico legítimo se preserva: `GET /` y `GET /actuator/health` siguen respondiendo 200.

5.2 Suite automatizada

`pre-analysis/tests/02-post-waf-attacks.sh` (espejo del test pre-WAF con las assertions invertidas) verifica bloqueos y que la app legítima siga viva:

5.3 Defensa en profundidad confirmada en el audit log

Origen	Regla(s)	Qué cazó
Custom	99001	/actuator/{info,metrics,prometheus,env}
Custom	99002	traversal .. / %2e%2e sobre /proxy/*
Custom	99003	/proxy/<svc>/actuator (ruta admin vía proxy)
Custom	99004/99005	/proxy/* sin cookie SESSIONID válida (acceso directo curl/scanner)
Custom	99010	User-Agents de sqlmap / nikto / nuclei
OWASP CRS	941100/110/160/390	XSS (libinjection + filtros) en checkout
OWASP CRS	942100/350	SQL injection (libinjection) en checkout
OWASP CRS	949110	Anomaly score excedido (suma de scores parciales)

6. Problemas encontrados durante el tuning

Lo que costó hacer funcionar — y cómo se resolvió. Esta sección es la más honesta del trabajo y resume el aprendizaje real.

- Comillas simples rompían el parseo de nginx.** `ingress-nginx` envuelve el `modsecurity-snippet` en `modsecurity_rules '...'` (comillas simples). Cualquier `'` dentro de una regla (p. ej. `msg:'...'`) cerraba el string y rompía nginx. **Solución:** mover las reglas custom a un archivo aparte (`rules/the-store.conf`) montado como volumen y cargado con `Include`; en un `Include` ModSecurity parsea directo y las comillas conviven sin problema.
- libmodsecurity v3 es quisquilloso con las continuaciones de línea.** Las reglas multi-línea con `\` a veces cortaban la lista de acciones a la mitad. **Solución:** una `SecRule` por línea física (largas pero a prueba de balas).
- Falso positivo del CRS sobre Host numérico.** En el POC local el `Host` es `127.0.0.1/localhost`, y la regla CRS `920350` ("Host header is a numeric IP address") disparaba sobre tráfico legítimo. **Solución:** regla custom `99020` que hace `ctl:ruleRemoveById=920350` **sólo** cuando el `Host` es `localhost` — una whitelist quirúrgica, no un apagado global.
- El rate limit "no funcionaba"... hasta mandar requests concurrentes.** Un loop secuencial de curls es demasiado lento (cada request tarda) y queda por debajo de 10 rps, así que nunca dispara el `429`. **Solución:** lanzar el burst con concurrencia real (`xargs -P 50`) para agotar el burst (`10·3=30`) y throttlear el resto.
- El traversal daba 404 en vez de 403.** `curl` normaliza el `../` del lado cliente antes de enviarlo, así que el `..` literal nunca llegaba al WAF. **Solución:** usar `curl --path-as-is` para que el `..` viaje crudo y dispare la regla `99002`.

6. **No romper la UI con la CSP.** Una `Content-Security-Policy` estricta rompía estilos/scripts inline de la UI. **Solución:** entregarla en modo **Report-Only** (observa y reporta violaciones sin bloquear); el clickjacking igual queda cubierto por `X-Frame-Options`, que sí aplica.
 7. **No tumbar el pod del UI.** Un bloqueo ciego de `/actuator/*` también bloqueaba `/actuator/health`, que Kubernetes usa para los probes de readiness/liveness — bloquearlo reiniciaría el pod en loop. **Solución:** la regla `99001` excluye explícitamente `health`.
 8. **Persistencia del audit log.** El controller tiene `readOnlyRootFilesystem` en varios paths; `/tmp` siempre es escribible. **Solución:** escribir el audit log en `/tmp/modsec_audit.log` y extraerlo con `kubectrl exec ... tail` para la demo, sin montar un volumen extra.
-

7. Limitaciones honestas y trabajo futuro

- **Un WAF no reemplaza código seguro.** Es defensa en profundidad, no una bala de plata. Las vulnerabilidades de fondo (el `ProxyController` sin sanitizar, la validación parcial del checkout) siguen en el código; el WAF las contiene en el perímetro.
- **Sólo cubre el tráfico norte-sur.** El tráfico este-oeste entre pods (`ui` → `catalog/carts/...`) no pasa por el Ingress, así que el WAF no lo ve. Mitigarlo requeriría un service mesh (mTLS + sidecars) o NetworkPolicies.
- **Bypass por encoding.** Doble/triple URL-encode, Unicode raro o JSON con keys inesperadas son un campo activo de evasión de WAFs basados en firmas.
- **Falsos positivos a Paranoia Levels altos.** Quedamos en PL1 para la demo; PL2+ requeriría una ronda de tuning más larga sobre los endpoints legítimos.
- **HTTPS termina en el WAF.** Si la terminación TLS estuviera antes (otro proxy), el WAF no vería el contenido. En este POC TLS termina en el Ingress, así que OK.
- **CSP en Report-Only** observa pero no bloquea; pasar a enforcement requiere validar antes que la UI no rompa.
- **Ataques de lógica de negocio** (abusar de un cupón N veces) un WAF de reglas no los detecta; requieren reglas de negocio o bot management.

Trabajo futuro: subir a PL2 con tuning, evaluar **Coraza** (sucesor de ModSec en Go, compatible con CRS), agregar NetworkPolicies para el este-oeste y enforcement real de la CSP.

8. Conclusión

El POC cumple el criterio de éxito de la pre-entrega: **los 10 vectores hoy explotables en el punto de entrada HTTP quedaron detectados o bloqueados por el WAF, sin romper el acceso normal** a la home, el catálogo, el carrito ni el checkout (`27/27` tests post-WAF en verde, `/` y `/actuator/health` siguen en `200`).

La decisión arquitectural clave — **ModSecurity + CRS en el Ingress, con reglas custom sólo para la lógica propia de The Store** — se validó en la práctica: el CRS resolvió las inyecciones genéricas (XSS/SQLi) sin una sola línea custom, mientras que las 7 reglas propias cubrieron lo que el CRS no conoce (Actuator, proxy traversal, scanners). Todo quedó **declarativo, idempotente y reversible**, integrado en `local.sh` y reproducible desde cero siguiendo el [HOWTO](#).

El mayor aprendizaje no fue escribir reglas, sino **operar el WAF**: el tuning (comillas, falsos positivos, no tumbar los probes, no romper la UI) es donde se juega que un WAF sea útil en vez de un estorbo.